Spring 2026

CS 5757: Optimization Methods for Robotics
Homework 3: Optimal Control

Due: Thursday, March 26, 2026, at 11:59pm on Gradescope, Total Points: 50

Learning goals for this problem set:

- **Problem 1:** Study how LQR and iLQR perform under noise, and understand the benefits of replanning.
- **Problem 2:** Explore using Sequential Convex Programming (SCP) to impose collision avoidance constraints.
- **Problem 3:** Read ALTRO; compare and contrast using AL methods to impose constraints with SQP.
- **Problem 4:** Use SCP-based MPC to track a trajectory under model mismatch; understand the role of trust regions, replanning horizon, and constraint handling.

Submission Instructions: Submit your writeup as a PDF to the *Written Assignment* on Gradescope. For the *Programming Assignment*, submit *only* your completed `solutions.py`.
You are **strongly encouraged** to typeset your homework submissions using LaTeX(if you do not know LaTeXyet, now is as good a time as any to learn it – you will need it in grad school anyway). If you opt for handwritten submissions, make sure that they are legible – if we cannot read it, we cannot grade it!

3.1 (i)LQR for Stochastic Systems

Recall the finite-horizon LQR problem with quadratic costs and linear dynamics:

$$\begin{aligned} \min_{\boldsymbol{u}_{0:T-1}} \quad & \frac{1}{2}\boldsymbol{x}_T^\top \boldsymbol{Q}_T \boldsymbol{x}_T + \sum_{t=0}^{T-1}\left(\frac{1}{2}\boldsymbol{x}_t^\top \boldsymbol{Q}_t \boldsymbol{x}_t + \frac{1}{2}\boldsymbol{u}_t^\top \boldsymbol{R}_t \boldsymbol{u}_t\right) \\ \text{s.t.} \quad & \boldsymbol{x}_{t+1} = \boldsymbol{A}\boldsymbol{x}_t + \boldsymbol{B}\boldsymbol{u}_t. \end{aligned}$$

Now suppose the dynamics are subject to additive stochastic noise,

$$\boldsymbol{x}_{t+1} = \boldsymbol{A}\boldsymbol{x}_t + \boldsymbol{B}\boldsymbol{u}_t + \boldsymbol{w}_t, \qquad \boldsymbol{w}_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\boldsymbol{0}, \boldsymbol{\Sigma}),$$

and the objective becomes minimizing the *expected* cumulative cost.

(a) Using dynamic programming, derive the optimal policy $\boldsymbol{u}_t^*(\boldsymbol{x}_t)$ by working backward from the terminal cost. Show that the value function takes the form

$$V_t(\boldsymbol{x}) = \frac{1}{2}\,\boldsymbol{x}^\top \boldsymbol{P}_t\,\boldsymbol{x} + d_t,$$

where $\boldsymbol{P}_t$ satisfies the same Riccati recursion as the deterministic case,

$$\boldsymbol{P}_t = \boldsymbol{Q}_t + \boldsymbol{A}^\top \boldsymbol{P}_{t+1}\boldsymbol{A} - \boldsymbol{A}^\top \boldsymbol{P}_{t+1}\boldsymbol{B}\big(\boldsymbol{R}_t + \boldsymbol{B}^\top \boldsymbol{P}_{t+1}\boldsymbol{B}\big)^{-1}\boldsymbol{B}^\top \boldsymbol{P}_{t+1}\boldsymbol{A},$$

and the optimal control $\boldsymbol{u}_t^* = -\boldsymbol{K}_t\,\boldsymbol{x}_t$ with $\boldsymbol{K}_t = (\boldsymbol{B}^\top \boldsymbol{P}_{t+1}\boldsymbol{B} + \boldsymbol{R}_t)^{-1}\boldsymbol{B}^\top \boldsymbol{P}_{t+1}\boldsymbol{A}$ is identical to the deterministic case. Derive an expression for d_t in terms of $\boldsymbol{P}_t$ and the noise covariance $\boldsymbol{\Sigma}$, and explain why the noise does not affect the optimal policy.

Hint: Apply the dynamic programming recursion from lecture, taking the expectation over $\boldsymbol{w}_t$ and using $\mathbb{E}[\boldsymbol{w}_t] = \boldsymbol{0}$.

For the remaining parts, we apply iLQR to the **acrobot**—an inverted double pendulum with a single actuator at the elbow.

(b) Given nonlinear dynamics $\boldsymbol{x}_{t+1} = \boldsymbol{f}(\boldsymbol{x}_t, \boldsymbol{u}_t)$ and running cost $c(\boldsymbol{x}_t, \boldsymbol{u}_t)$, iLQR linearizes the dynamics and quadraticizes the cost around a nominal trajectory $(\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t)$:

$$\begin{aligned}
\boldsymbol{f}(\boldsymbol{x}_t, \boldsymbol{u}_t) &\approx \boldsymbol{f}(\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t) + \boldsymbol{A}_t\,\delta\boldsymbol{x}_t + \boldsymbol{B}_t\,\delta\boldsymbol{u}_t, \\
c(\boldsymbol{x}_t, \boldsymbol{u}_t) &\approx c(\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t) + \boldsymbol{q}_t^\top \delta\boldsymbol{x}_t + \boldsymbol{r}_t^\top \delta\boldsymbol{u}_t + \tfrac{1}{2}\delta\boldsymbol{x}_t^\top \boldsymbol{Q}_t\,\delta\boldsymbol{x}_t + \tfrac{1}{2}\delta\boldsymbol{u}_t^\top \boldsymbol{R}_t\,\delta\boldsymbol{u}_t + \delta\boldsymbol{x}_t^\top \boldsymbol{S}_t\,\delta\boldsymbol{u}_t,
\end{aligned}$$

where $\delta\boldsymbol{x}_t = \boldsymbol{x}_t - \bar{\boldsymbol{x}}_t$, $\delta\boldsymbol{u}_t = \boldsymbol{u}_t - \bar{\boldsymbol{u}}_t$, and $\boldsymbol{A}_t, \boldsymbol{B}_t$ are dynamics Jacobians and $\boldsymbol{Q}_t, \boldsymbol{R}_t, \boldsymbol{S}_t, \boldsymbol{q}_t, \boldsymbol{r}_t$ are cost Hessians and gradients evaluated at $(\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t)$. Implement `linearize(f, cost, s, a)` using JAX autodiff, returning $(\boldsymbol{Q}_t, \boldsymbol{R}_t, \boldsymbol{S}_t, \boldsymbol{q}_t, \boldsymbol{r}_t, \boldsymbol{A}_t, \boldsymbol{B}_t)$ where $\boldsymbol{S}_t = \frac{\partial^2 c}{\partial \boldsymbol{x}\,\partial \boldsymbol{u}}$.

(c) Implement `ilqr_backward_step(P, p, Q, R, S, q, r, A, B, reg)` $\to (\boldsymbol{P}_t, \boldsymbol{p}_t, \boldsymbol{K}, \boldsymbol{k})$. Given the value function $V_{t+1}(\delta\boldsymbol{x}) \approx \frac{1}{2}\delta\boldsymbol{x}^\top \boldsymbol{P}\,\delta\boldsymbol{x} + \boldsymbol{p}^\top \delta\boldsymbol{x}$ and the expansions from part (c), compute the action-value expansions $\boldsymbol{H}_{xx}, \boldsymbol{H}_{xu}, \boldsymbol{H}_{uu}, \boldsymbol{h}_x, \boldsymbol{h}_u$, solve for the optimal feedback gains $(\boldsymbol{K}, \boldsymbol{k})$, and propagate the value function backward to $(\boldsymbol{P}_t, \boldsymbol{p}_t)$.

Hint: Apply Levenberg–Marquardt regularization $\boldsymbol{H}_{uu} \leftarrow \boldsymbol{H}_{uu} + \texttt{reg}\cdot\boldsymbol{I}$ and use `jnp.linalg.solve` rather than an explicit inverse.

(d) Implement `simulate` in `solution.py`, which rolls out the system under additive Gaussian noise. Complete the two indicated lines to apply either the open-loop or closed-loop iLQR control at each timestep, depending on the bool `use_feedback`.

(e) Run `acrobot_ilqr.py`, which simulates the acrobot under both open-loop and closed-loop iLQR across increasing noise levels. Include the generated plot in your writeup.

(i) At low noise, how does closed-loop performance compare to open-loop? Why should we expect this from part (a)?

(ii) As noise increases, the closed-loop policy eventually degrades sharply. What assumption underlying the iLQR feedback gains is violated when the noise is large?

Solution:

(a) We will prove this using induction, and the principle of optimality. Assume that

$$V_{t+1}(\boldsymbol{x}) = \tfrac{1}{2}\,\boldsymbol{x}^\top \boldsymbol{P}_{t+1}\,\boldsymbol{x} + d_{t+1}$$

for some $\boldsymbol{P}_{t+1} \succeq 0$ and scalar d_{t+1}. The base case is $\boldsymbol{P}_T = \boldsymbol{Q}_T$, $d_T = 0$.

By the principle of optimality, the Bellman equation at time t is

$$V_t(\boldsymbol{x}_t) = \min_{\boldsymbol{u}} \left\{ \frac{1}{2}\, \boldsymbol{x}_t^\top \boldsymbol{Q}_t\, \boldsymbol{x}_t + \frac{1}{2}\, \boldsymbol{u}^\top \boldsymbol{R}_t\, \boldsymbol{u} + \mathbb{E}_{\boldsymbol{w}_t} \left[\frac{1}{2} \left(\boldsymbol{A}\boldsymbol{x}_t + \boldsymbol{B}\boldsymbol{u} + \boldsymbol{w}_t \right)^\top \boldsymbol{P}_{t+1} \left(\boldsymbol{A}\boldsymbol{x}_t + \boldsymbol{B}\boldsymbol{u} + \boldsymbol{w}_t \right) + d_{t+1} \right] \right\}.$$

Expanding the quadratic and taking the expectation, all terms independent of $\boldsymbol{u}$ drop from the $\arg\min$, and all terms linear in $\boldsymbol{w}_t$ vanish since $\mathbb{E}[\boldsymbol{w}_t] = \boldsymbol{0}$. The minimization reduces to

$$\boldsymbol{u}_t^* = \arg\min_{\boldsymbol{u}} \left\{ \boldsymbol{u}^\top \boldsymbol{B}^\top \boldsymbol{P}_{t+1}\, \boldsymbol{A}\, \boldsymbol{x}_t + \frac{1}{2}\, \boldsymbol{u}^\top \left(\boldsymbol{B}^\top \boldsymbol{P}_{t+1}\, \boldsymbol{B} + \boldsymbol{R}_t \right) \boldsymbol{u} \right\}.$$

Setting the gradient to zero and solving:

$$\boldsymbol{u}_t^* = -\underbrace{\left(\boldsymbol{B}^\top \boldsymbol{P}_{t+1}\, \boldsymbol{B} + \boldsymbol{R}_t \right)^{-1} \boldsymbol{B}^\top \boldsymbol{P}_{t+1}\, \boldsymbol{A}}_{\boldsymbol{K}_t}\, \boldsymbol{x}_t.$$

Substituting $\boldsymbol{u}_t^*$ back into the Bellman equation and collecting terms, the value function at time t takes the form $V_t(\boldsymbol{x}_t) = \frac{1}{2}\, \boldsymbol{x}_t^\top \boldsymbol{P}_t\, \boldsymbol{x}_t + d_t$, where

$$\begin{aligned} \boldsymbol{P}_t &= \boldsymbol{Q}_t + \boldsymbol{A}^\top \boldsymbol{P}_{t+1}\, \boldsymbol{A} - \boldsymbol{A}^\top \boldsymbol{P}_{t+1}\, \boldsymbol{B} \left(\boldsymbol{R}_t + \boldsymbol{B}^\top \boldsymbol{P}_{t+1}\, \boldsymbol{B} \right)^{-1} \boldsymbol{B}^\top \boldsymbol{P}_{t+1}\, \boldsymbol{A}, \\ d_t &= d_{t+1} + \tfrac{1}{2}\, \mathrm{tr}(\boldsymbol{P}_{t+1}\, \boldsymbol{\Sigma}), \end{aligned}$$

which finishes the induction. Since the noise enters the value function only through the additive scalar d_t, it never couples with $\boldsymbol{u}$, and both $\boldsymbol{K}_t$ and $\boldsymbol{P}_t$ are identical to the deterministic case.

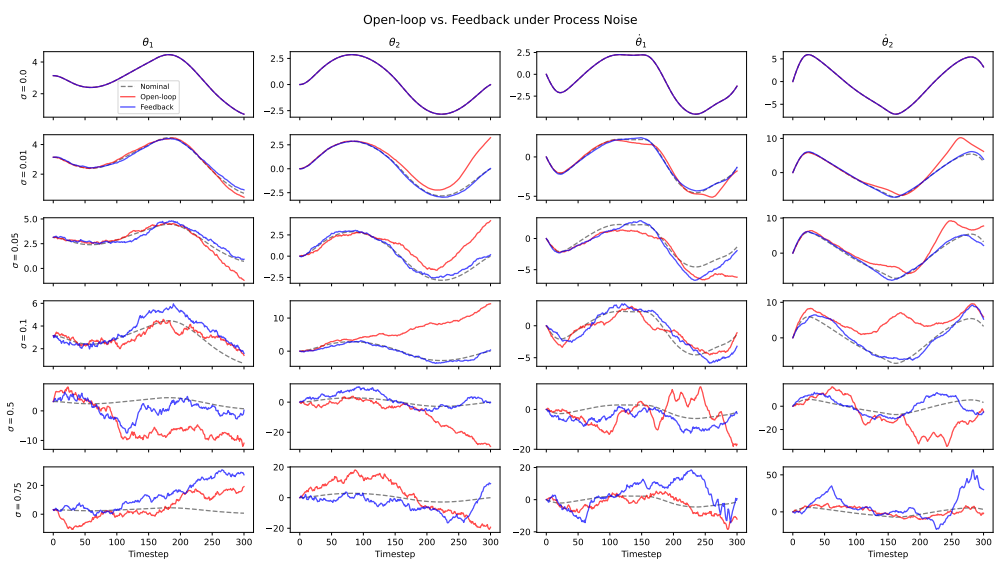

Figure 1: Plots comparing open- and closed-loop control for the acrobot with iLQR.

(e) Figure 1 plots the open- and closed-loop rollouts for iLQR for the acrobot swingup. At low noise, we notice that the performance is identical - this is expected, since the robot dynamics will match those exactly that we used to compute the open-loop control. As the noise grows, the robot's trajectory will start to diverge from the planned trajectory, which means the feedback

can stabilize the robot to the planned trajectory and reject the noise.

When the noise grows too large, however, the feedback gain will become invalid because the linearized system used to compute the gains will be a poor approximation for the dynamics in the new region. Thus, we can expect the closed-loop performance to also degrade sharply with the invalid linearized dynamics.

3.2 Sequential Convex Programming

Consider a drone navigating through n_{obs} spherical obstacles, each with center $\boldsymbol{o}_i \in \mathbb{R}^3$ and radius r_i. The drone has radius r_d and state $\boldsymbol{x} = [\boldsymbol{p}^\top, \boldsymbol{v}^\top]^\top \in \mathbb{R}^6$. We want to find a collision-free trajectory minimizing

$$\min_{\boldsymbol{u}_{0:T-1}} \quad \frac{1}{2}(\boldsymbol{x}_T - \boldsymbol{x}_g)^\top \boldsymbol{P}\,(\boldsymbol{x}_T - \boldsymbol{x}_g) + \sum_{t=0}^{T-1} \left[\frac{1}{2}(\boldsymbol{x}_t - \boldsymbol{x}_g)^\top \boldsymbol{Q}\,(\boldsymbol{x}_t - \boldsymbol{x}_g) + \frac{1}{2}\boldsymbol{u}_t^\top \boldsymbol{R}\,\boldsymbol{u}_t\right] \tag{1}$$

$$\text{s.t.} \quad \boldsymbol{x}_{t+1} = \boldsymbol{f}(\boldsymbol{x}_t, \boldsymbol{u}_t), \tag{2}$$

$$d_i(\boldsymbol{x}_t) \geq 0, \quad i = 1, \ldots, n_{\text{obs}}, \quad t = 0, \ldots, T, \tag{3}$$

which is nonconvex due to the nonlinear dynamics (2) and obstacle constraints (3). SCP solves this by iteratively linearizing both around the current nominal trajectory and solving the resulting convex QP, whose solution becomes the new nominal trajectory.
Parts (f) and (g) use CVXPY; see here for basic usage. Note that we will be using `cvx.Parameter`, which allows you to update problem data between solves without reconstructing the problem.

(a) Implement `signed_distance(state)`, where `state` has shape `(nx,)`. Return $\boldsymbol{d} \in \mathbb{R}^{n_{\text{obs}}}$, $d_i = \|\boldsymbol{p} - \boldsymbol{o}_i\|_2 - r_i - r_d$, using global constants `OBSTACLE_CENTERS (n_obs, 3)`, `OBSTACLE_RADII (n_obs,)`, and `DRONE_RADIUS`.

(b) The signed distance constraint $d_i(\boldsymbol{x}) = \|\boldsymbol{p} - \boldsymbol{o}_i\|_2 - r_i - r_d$ is nonconvex (the inequality is backwards), so we linearize the obstacle constraint $d_i(\boldsymbol{x}) \geq 0$ about a nominal state $\bar{\boldsymbol{x}}$:

$$\underbrace{\nabla_{\boldsymbol{x}} d_i(\bar{\boldsymbol{x}})^\top}_{\boldsymbol{g}_i}\, \boldsymbol{x} \;+\; \underbrace{d_i(\bar{\boldsymbol{x}}) - \nabla_{\boldsymbol{x}} d_i(\bar{\boldsymbol{x}})^\top \bar{\boldsymbol{x}}}_{h_i} \;\geq\; 0.$$

Show that any $\boldsymbol{x}$ satisfying this linearized constraint is *guaranteed* to satisfy $d_i(\boldsymbol{x}) \geq 0$. Does this hold for the signed-distance of a non-convex shape?

Hint: Prove that d_i is convex using the definition from class.

(c) Linearize $\boldsymbol{x}_{t+1} = \boldsymbol{f}(\boldsymbol{x}_t, \boldsymbol{u}_t)$ around $(\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t)$ and show that the result can be written as $\boldsymbol{x}_{t+1} \approx \boldsymbol{A}_t \boldsymbol{x}_t + \boldsymbol{B}_t \boldsymbol{u}_t + \boldsymbol{c}_t$. Derive $\boldsymbol{A}_t$, $\boldsymbol{B}_t$, and $\boldsymbol{c}_t$.

(d) Implement `affinize_dynamics(f, s_bar, u_bar)` using JAX autodiff, returning $(\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{c})$ as derived in part (c). Inputs `s_bar (nx,)` and `u_bar (nu,)`; outputs have shapes `(nx, nx)`, `(nx, nu)`, `(nx,)`.

You should also implement `affinize_constraint(g, s_bar)`, returning $(\boldsymbol{G}, h)$ where $\boldsymbol{G} = \frac{\partial \boldsymbol{d}}{\partial \boldsymbol{x}_t}$ and $h = g(\bar{\boldsymbol{x}}) - \boldsymbol{G}\bar{\boldsymbol{x}}$, following the same pattern as `affinize_dynamics`. Input `s_bar (nx,)`; outputs have shapes `(n_obs, nx)` and `(n_obs,)`.

(e) Complete `build_drone_scp`. The cost, dynamics constraints, and initial condition are already provided. Add the linearized obstacle constraints for all $t \in \{1, \ldots, N\}$:

$$\boldsymbol{G}_t\, \boldsymbol{x}_t + \boldsymbol{h}_t \geq 0,$$

using parameters `G_p (N+1, n_obs, nx)` and `h_p (N+1, n_obs)`.

(f) Implement `solve_scp`. At each iteration, use `affinize_dynamics` and `affinize_constraint` to compute $(\boldsymbol{A}_t, \boldsymbol{B}_t, \boldsymbol{c}_t)$ and $(\boldsymbol{G}_t, h_t)$ along the current nominal trajectory `s_nom (N+1, nx)`, `u_nom (N, nu)`, assign them to the CVXPY parameters `(A_p, B_p, c_p) (N, nx, nx)`, `(N, nx, nu)`, `(N, nx)` and `(G_p, h_p) (N+1, n_obs, nx)`, `(N+1, n_obs)`, and solve. Run `problem2.py` and include the 3D trajectory plot in your writeup.

(g) In part (b) you showed that the linearized obstacle constraints are *conservative*: satisfying them guarantees collision avoidance at every SCP iteration. Does the same hold for the linearized dynamics? What does this asymmetry imply about the dynamic feasibility of an unconverged solution?

Solution:

(b) The signed distance d_i is the ℓ_2 norm shifted by a constant. For any $\alpha \in [0, 1]$ and states $\boldsymbol{x}, \boldsymbol{y}$:

$$\begin{aligned} d_i(\alpha \boldsymbol{x} + (1-\alpha)\boldsymbol{y}) &= \|\alpha(\boldsymbol{p}_x - \boldsymbol{o}_i) + (1-\alpha)(\boldsymbol{p}_y - \boldsymbol{o}_i)\|_2 - (r_i + r_d) \\ &\leq \alpha\|\boldsymbol{p}_x - \boldsymbol{o}_i\|_2 + (1-\alpha)\|\boldsymbol{p}_y - \boldsymbol{o}_i\|_2 - (r_i + r_d) \\ &= \alpha\, d_i(\boldsymbol{x}) + (1-\alpha)\, d_i(\boldsymbol{y}), \end{aligned}$$

where the inequality follows from the triangle inequality and homogeneity of norms. Thus d_i is convex.

By the first-order condition of convexity, $d_i(\bar{\boldsymbol{x}}) + \nabla d_i(\bar{\boldsymbol{x}})^\top(\boldsymbol{x} - \bar{\boldsymbol{x}}) \leq d_i(\boldsymbol{x})$ for all $\boldsymbol{x}$. Therefore any $\boldsymbol{x}$ satisfying the linearized constraint has $d_i(\boldsymbol{x}) \geq d_i(\bar{\boldsymbol{x}}) + \nabla d_i(\bar{\boldsymbol{x}})^\top(\boldsymbol{x} - \bar{\boldsymbol{x}}) \geq 0$, so the true constraint holds as well.

This "conservatism" does not hold for all obstacles - if we wanted to apply this to general shapes we would need to use a tool like "convex decomposition" to approximate them with collections of convex shapes.

(c) The first-order Taylor expansion of $\boldsymbol{f}$ around $(\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t)$ gives

$$\boldsymbol{x}_{t+1} \approx \boldsymbol{f}(\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t) + \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t} (\boldsymbol{x}_t - \bar{\boldsymbol{x}}_t) + \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}\right|_{\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t} (\boldsymbol{u}_t - \bar{\boldsymbol{u}}_t),$$

which yields $\boldsymbol{x}_{t+1} \approx \boldsymbol{A}_t\boldsymbol{x}_t + \boldsymbol{B}_t\boldsymbol{u}_t + \boldsymbol{c}_t$ for

$$\boldsymbol{A}_t = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t}, \quad \boldsymbol{B}_t = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{u}}\right|_{\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t}, \quad \boldsymbol{c}_t = \boldsymbol{f}(\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t) - \boldsymbol{A}_t\bar{\boldsymbol{x}}_t - \boldsymbol{B}_t\bar{\boldsymbol{u}}_t.$$

(f) Figure 2 shows a visualization of the drone trajectory. We can see it chooses to go below the obstacles, to approach the goal, but the resulting trajectory is collision-free.

(g) The dynamics constraints are not conservative - the linearized dynamics are only approximations of the true nonlinear dynamics. At convergence (where the SCP iterations return no change), the dynamics constraints will be exactly satisfied, but until then, the dynamics are only approximate. Thus, the trajectories returned by SCP may be "optimistic" compared to the full nonlinear ones.

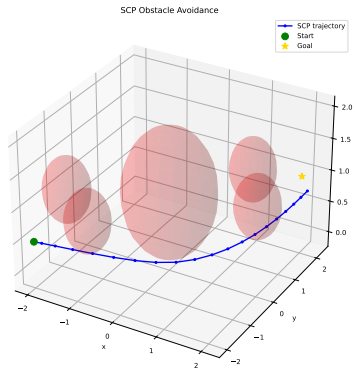

Figure 2: Visualization of the SCP trajectory.

3.3 ALTRO

For this question, we will read *ALTRO: A Fast Solver for Constrained Trajectory Optimization* by Howell et al. The paper extends iLQR to handle constraints via an augmented Lagrangian method.

(a) Why can't iLQR handle state or input constraints (e.g., joint limits, obstacle avoidance)?

(b) Does ALTRO guarantee constraint satisfaction at each iLQR iteration, or only upon convergence of the augmented Lagrangian outer loop? Are the trajectories estimated by ALTRO dynamically feasible at each iteration? How does this compare to the SQP approach you implemented in the previous problem?

(c) What is the role of the regularization parameter ρ in the δu update given by Equation (16)? Why is this needed in practice for general trajectory optimization problems?

(d) The authors propose an "infeasible start" method in Section III.B. Give a concrete robotics scenario where this would be practically useful, and explain why a standard (dynamically feasible) initialization would struggle in that setting.

(e) Suppose our dynamics involve intermittent contact (e.g., a humanoid walking across the ground), so that the dynamics gradients $\partial f/\partial x$, $\partial f/\partial u$ are unreliable or discontinuous. How would you expect ALTRO's iLQR backward pass to behave? Be specific about which computational step breaks down.

(f) What is the fundamental limitation of enforcing constraints via a pure penalty method (adding $\rho\|c(x,u)\|^2$ to the cost) inside iLQR, and how does the Augmented Lagrangian formulation resolve it? Your answer should address both constraint satisfaction and numerical conditioning of the Riccati backward pass.

Solution:

(a) The iLQR "backwards pass" uses the typical Riccati recursion for unconstrained LQR, which does not account for any constraints other than dynamics and initial condition. Many of the constraints we care about are inequalities, which would not admit the same structured "backward pass" – this breaks the structure iLQR is leveraging to derive the gain updates.

(b) ALTRO does not guarantee constraint satisfaction in each iteration, only at the end of the AL procedure. Since AL can converge slowly, this "eventual" feasibility is a downside of using

ALTRO compared to other solvers that may have faster convergence to constraint satisfaction.

On the other hand, it does guarantee dynamic feasibility (since the iLQR trajectories are "roll-outs" peformed with the true dynamics).

These properties are the inverse of the SCP approach in the previous problem - each SCP iteration will guarantee satisfaction of the convex obstacle constraints, but we can only guarantee dynamic feasibility at convergence.

(c) For general non-convex costs and dynamics, the Q-function Hessian Q_{uu} need not be positive definite or invertible. Adding a positive ρ can make the resulting matrix invertible, at the cost of losing curvature information in the δu update that could speed convergence.

(d) Even for simple problems like acrobot swingup, it is difficult to provide an "initial guess" for the optimal controls, and more natural to provide a rough guess for the optimal trajectory (e.g., interpolating between the start and goal).

Allowing iLQR to use an infeasible start enables us to provide this initial state information without it needing to be dynamically feasible or to solve for the resulting control sequence.

(e) For non-differentiable phenomena like contact, the dynamics gradients are very poor approximations, even locally, for the system, meaning that iLQR backwards pass is not incorporating the full dynamics information (e.g., the effects of contact) into the backwards pass. In particular, the dynamics gradients are only valid for the current contact mode (i.e., they assume the foot of the humanoid cannot make or break contact), meaning that if the true optimal control would be to change contact modes, it is incable of incorporating this information into its update.

(f) A pure penalty method requires $\rho \to \infty$ to achieve exact constraint satisfaction — the constraint violation only vanishes asymptotically. But as ρ grows, the cost Hessian becomes increasingly ill-conditioned, since the penalty term dominates and creates a poorly-scaled Q_{uu} matrix, destabilizing the Riccati backward pass.

The Augmented Lagrangian resolves this by maintaining multiplier estimates λ that are updated after each iLQR solve via $\lambda \leftarrow \lambda + \rho\, c(x, u)$. These multipliers form a "first-order" penalty that encourages feasibility, and can provide exact feasibility with finite ρ. The Hessian therefore remains well-conditioned for each iteration, which, especially for numerically sensitive operations like the Riccati recursion, can significantly improve convergence in practice.

3.4 Model Predictive Control

Even a well-optimized trajectory can fail in practice when the planning model differs from the real system. Here, we plan using a simplified RK4 model of the acrobot but execute on a full MuJoCo simulator with joint damping and perturbed inertial parameters. Under open-loop execution, small model errors accumulate over the horizon and the trajectory degrades. Model Predictive Control (MPC) addresses this by replanning at each step: we solve an SCP subproblem from the current observed state, execute only the first few controls, then replan from the new state. This closed-loop structure lets the planner continually correct for model mismatch rather than committing to a single fixed plan.

Because linearization is only accurate near the nominal trajectory, each SCP subproblem restricts the solution to a *trust region* — a neighborhood of the current nominal trajectory within which the linearized model is a reliable approximation. Parts (a) and (b) analyze the ℓ_2 trust region subproblem theoretically, since it is easier to reason about. For the MPC implementation, we use an ℓ_∞ trust region $\|\boldsymbol{d}\|_\infty \leq \rho$, which reduces to box constraints and can be solved as a standard QP without second-order cone programming.

(a) Ignoring the dynamics and other constraints, the SCP subproblem at each iteration is a quadratic program with a trust region constraint:

$$\begin{aligned} \min_{\boldsymbol{d}} \quad & \frac{1}{2}\boldsymbol{d}^\top \boldsymbol{H}\, \boldsymbol{d} + \boldsymbol{g}^\top \boldsymbol{d} \\ \text{s.t.} \quad & \|\boldsymbol{d}\|_2^2 \leq \Delta^2, \end{aligned}$$

where $\boldsymbol{H}$ is the (possibly indefinite) Hessian of the linearized cost, $\boldsymbol{g}$ is the cost gradient, and $\Delta > 0$ is the trust region radius. Write the KKT conditions for this problem and show that the optimal $\boldsymbol{d}^*$ satisfies, for some $\mu^* \geq 0$:

$$(\boldsymbol{H} + \mu^* \boldsymbol{I})\, \boldsymbol{d}^* = -\boldsymbol{g}.$$

(b) Show that if $\boldsymbol{H}$ has a negative eigenvalue, the unconstrained quadratic program

$$\min_{\boldsymbol{d}} \quad \frac{1}{2}\boldsymbol{d}^\top \boldsymbol{H}\, \boldsymbol{d} + \boldsymbol{g}^\top \boldsymbol{d}$$

is unbounded below. What does this imply about solving SCP subproblems without a trust region when the Hessian is indefinite?

Remark: It is easy to show that when $\boldsymbol{H}$ is indefinite, the trust region constraint must be active ($\mu^* > 0$). From part (a), we can see this means that μ^* is implicitly acting as regularization.

(c) Complete `build_acrobot_scp` in `solution.py`, which constructs a CVXPY problem for one SCP subproblem. The cost function and affine dynamics constraints are already provided. The decision variables are `s`, shape `(N+1, nx)`, and `u`, shape `(N, nu)`, and the nominal trajectory is given by the parameters `s_bar_p`, shape `(N+1, nx)`, and `u_bar_p`, shape `(N, nu)`. Add the following constraints:

- **Control bounds** for all $t \in \{0, \ldots, N-1\}$:

$$-u_{\max} \;\leq\; u_t \;\leq\; u_{\max}.$$

- ℓ_∞ **trust regions** for all $t \in \{0, \dots, N-1\}$:

$$\|u_t - \bar{u}_t\|_\infty \le \rho, \qquad \|s_t - \bar{s}_t\|_\infty \le \rho \quad (t > 0),$$

  where the state trust region skips $t = 0$ since s_0 is already fixed by the initial condition. The trust region radius ρ and bound $u_{\max}$ are given by `trust_region` and `u_bound`, respectively.

(d) ✎ Run `problem3.py`. Include the generated plot in your writeup. The open-loop SCP plan fails on MuJoCo while receding-horizon MPC succeeds. Why does replanning at each step outperform using the fixed iLQR feedback gains?

(e) ✎ Reduce the MPC horizon to $N = 5$ and rerun. Include the resulting plot in your writeup. The acrobot fails to swing up. Why does MPC suddenly fail with a short horizon?

(f) ✎ Propose two modifications to the MPC formulation that could resolve the short-horizon problem. For each, describe the tradeoff it introduces.

Solution:

(a) The Lagrangian for this problem is given by

$$\mathcal{L}(\boldsymbol{d}, \mu) = \frac{1}{2}\boldsymbol{d}^\top \boldsymbol{H}\boldsymbol{d} + \boldsymbol{g}^\top \boldsymbol{d} + \frac{\mu}{2}(||\boldsymbol{d}||_2^2 - \Delta^2).$$

The KKT conditions for this problem are:

$$\begin{aligned} \nabla_{\boldsymbol{d}}\mathcal{L}(\boldsymbol{d}^*, \lambda) = \boldsymbol{H}\boldsymbol{d}^* + \boldsymbol{g} + \mu^*\boldsymbol{d}^* &= 0, \\ \frac{1}{2}||\boldsymbol{d}^*||_2^2 &\le \frac{\Delta^2}{2}, \\ \mu^* &\ge 0, \\ \frac{\mu^*}{2}(||\boldsymbol{d}^*||_2^2 - \Delta^2) &= 0 \end{aligned}$$

Rearranging the stationarity condition gives us the needed expression,

$$(\boldsymbol{H} + \mu^* I)\boldsymbol{d}^* = -\boldsymbol{g}.$$

(b) Suppose $\boldsymbol{H}$ has a negative eigenvalue $\lambda < 0$, with corresponding eigenvector $\boldsymbol{v}$.

For the unconstrained problem, if we choose $\boldsymbol{d} = t\boldsymbol{v}$, the objective is given by $t^2\lambda + t\boldsymbol{g}^\top\boldsymbol{v}$. As we send $t \to \infty$, the strictly negative term $t^2\lambda$ will dominate the linear term, which sends the unconstrained objective $\to -\infty$. We call this sort of ill-posed problem "unbounded below."

(d) Figure 3 shows the comparison between the methods. Replanning for this problem tends to work well because we are able to replan from the true acrobot state under the MuJoCo dynamics, which provides implicit state feedback. Since the iLQR gains are computed using the original RK4 trajectory, and because acrobot is highly unstable, the iLQR gains quickly can become useless for the real system.

(e) When the MPC horizon is too short, the system becomes myopic. For acrobot in particular, the optimal solution requires "pumping in" energy by using large torques (which incur a large

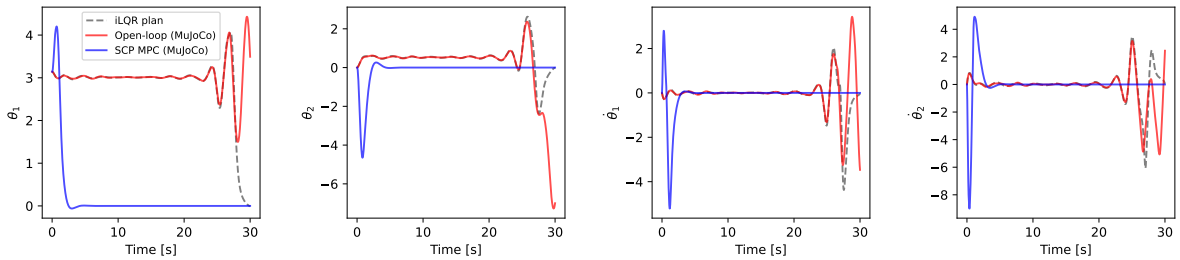

Figure 3: Comparison of the iLQR open-loop solution and the MPC solution.

cost) to begin a long swing to the origin. When the horizon is too short, the optimizer cannot "see" the end of the swing up (and its lower costs) and thus does not take these high-cost initial actions.

(f) The short-horizon problem can be fixed in several ways. Most common is to simply extend the horizon long enough that the optimizer can receive "proper credit" for its actions. This uses more compute but can result in better, non-myopic behavior.

A more elegant solution is to use a terminal cost $\boldsymbol{Q}_T$ in the MPC subproblem that matches the actual cost-to-go for the nonlinear dynamics. In theory, if this cost-to-go were exact, we could perform replanning by solving a one-step MPC problem.

3.5 AI Usage Survey

✎ (2 pts) Please describe how you used AI (if at all) while completing this homework. Note that you will not be penalized for AI usage, regardless of how extensively you used it, as long as you acknowledge it. We are generally curious about how we can improve the course material and better help with your learning!

References